

BN Lab

Samarth Bhatnagar

September 2026

Question 1

The chain-rule decomposition is useful for generating text because it allows the sentence to be generated one token at a time. At each step, the model predicts the probability distribution of the next token conditioned on the tokens that have already been generated. Thus, instead of generating the complete sentence at once, we repeatedly predict and generate the next token.

$$X_1 \sim P(X_1),$$

followed by

$$X_2 \sim P(X_2 | X_1),$$

$$X_3 \sim P(X_3 | X_1, X_2),$$

and so on. This allows an autoregressive model to generate a complete sequence by repeatedly predicting the next token.

Question2

The network assumes that the current word is conditionally independent of all previous words given the immediately preceding word. In probability notation,

$$\boxed{P(X_t | X_1, \dots, X_{t-1}) = P(X_t | X_{t-1})}$$

For example,

$$P(X_3 | X_1, X_2) = P(X_3 | X_2)$$

and

$$P(X_4 | X_1, X_2, X_3) = P(X_4 | X_3).$$

Thus, only the immediately preceding word is used to predict the next word.

Question 3

The conditional probability distribution is obtained using

$$P(w_j | w_i) = \frac{C(w_i, w_j)}{\sum_k C(w_i, w_k)},$$

where $C(w_i, w_j)$ is the number of times that w_j follows w_i .

For the given dataset:

$$P(\text{cat} \mid \text{the}) = \frac{3}{6} = \frac{1}{2},$$

$$P(\text{dog} \mid \text{the}) = \frac{3}{6} = \frac{1}{2}.$$

Therefore,

$$P(w \mid \text{the}) = \begin{cases} \frac{1}{2}, & w = \text{cat}, \\ \frac{1}{2}, & w = \text{dog}, \\ 0, & \text{otherwise.} \end{cases}$$

For the word “cat”:

$$P(\text{sat} \mid \text{cat}) = \frac{2}{3}, \quad P(\text{ran} \mid \text{cat}) = \frac{1}{3}.$$

Thus,

$$P(w \mid \text{cat}) = \begin{cases} \frac{2}{3}, & w = \text{sat}, \\ \frac{1}{3}, & w = \text{ran}, \\ 0, & \text{otherwise.} \end{cases}$$

For the word “dog”:

$$P(\text{sat} \mid \text{dog}) = \frac{2}{3}, \quad P(\text{ran} \mid \text{dog}) = \frac{1}{3}.$$

Thus,

$$P(w \mid \text{dog}) = \begin{cases} \frac{2}{3}, & w = \text{sat}, \\ \frac{1}{3}, & w = \text{ran}, \\ 0, & \text{otherwise.} \end{cases}$$

For the word “sat”:

$$P(\text{on} \mid \text{sat}) = 1.$$

For the word “ran”:

$$P(\text{to} \mid \text{ran}) = 1.$$

Hence,

$$P(w \mid \text{sat}) = \begin{cases} 1, & w = \text{on}, \\ 0, & \text{otherwise,} \end{cases}$$

and

$$P(w \mid \text{ran}) = \begin{cases} 1, & w = \text{to}, \\ 0, & \text{otherwise.} \end{cases}$$

There are zero-probability transitions for every word that was not observed as a successor of the corresponding current word.

Question 4

The transition counts should be stored in a data structure that maps each current token to the tokens that follow it and their corresponding counts. For example, conceptually,

$$\text{counts}[\text{current}][\text{next}] = C(\text{current}, \text{next}).$$

Thus, the program stores how many times each token follows another token.

Question 5

The conditional probability

$$P(X_t \mid X_{t-1})$$

is computed from the transition counts using

$$P(X_t = w \mid X_{t-1} = v) = \frac{C(v, w)}{\sum_u C(v, u)}$$

where $C(v, w)$ is the number of times w follows v in the training data.

Question 6

The program can choose the next word in two different ways.

In greedy generation, it always chooses the most probable word:

$$X_t = \arg \max_w P(w \mid X_{t-1}).$$

In sampling-based generation, it samples the next word according to the conditional probability distribution:

$$X_t \sim P(X_t \mid X_{t-1}).$$

Greedy generation is deterministic for a fixed context, whereas sampling is probabilistic and can produce different sequences from the same starting word.

Question 7

If the program encounters a word for which no transition has been observed, then there is no estimated conditional distribution for that word. Therefore, the program cannot select a next word using the learned transition probabilities.

A possible implementation can terminate generation, report that no transition was observed, or use a predefined fallback strategy. The exact behaviour depends on how the program is implemented.

Question 8

For every current word w , the conditional probabilities must satisfy

$$\sum_v P(v \mid w) = 1.$$

Therefore, if one of the totals is 0.87 instead of approximately 1.0, the implementation is incorrect. It indicates that the conditional probabilities have not been properly normalized, or that some probability mass has been lost.

Hence, the probability-normalization test can be used to detect an error in the implementation.

Question 9

No. The most probable prediction according to the model is not necessarily the word that a human would personally expect.

This is because the model makes predictions based only on the probability distribution learned from the training data. Human linguistic expectations can use much more context, semantic knowledge, and general understanding of language.

Thus, a probability model's prediction and human linguistic expectations do not necessarily coincide.

Question 10

Greedy generation always chooses the most probable next word:

$$X_t = \arg \max_w P(w \mid X_{t-1}).$$

Therefore, once a particular context is encountered, the same next word is always selected.

In sampling-based generation,

$$X_t \sim P(X_t \mid X_{t-1}),$$

so different words can be selected according to their probabilities.

Therefore, sampling produces more variation than greedy generation because it allows lower-probability words to be selected as well.

Question 11

The second-order model estimates

$$P(X_t \mid X_{t-2}, X_{t-1}),$$

whereas the first-order model estimates

$$P(X_t \mid X_{t-1}).$$

1. Graph structure

The first-order model has the structure

$$X_{t-1} \rightarrow X_t.$$

The second-order model has the structure

$$X_{t-2} \rightarrow X_t \leftarrow X_{t-1}.$$

2. Conditional probability table

The first-order model stores probabilities of the form

$$P(X_t \mid X_{t-1}),$$

while the second-order model stores probabilities of the form

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

Thus, the second-order model requires a larger conditional probability table because it considers pairs of previous tokens as contexts.

3. Amount of context

The first-order model uses one previous token, whereas the second-order model uses two previous tokens. Therefore, the second-order model has more context available for prediction.

4. Amount of data

The second-order model generally requires more training data because it must estimate probabilities for many possible pairs of previous tokens. With limited data, many contexts may never be observed, resulting in zero-probability contexts.

Question 12

Increasing the amount of context can improve prediction because the model has more information about the preceding sequence.

For example, instead of predicting using only

$$P(X_t \mid X_{t-1}),$$

the second-order model uses

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

The additional context can help distinguish between different possible next words.

However, increasing the context also increases the number of possible contexts and therefore increases the size of the conditional probability table. More training data is consequently required to observe these contexts and estimate their probabilities reliably.

Thus, increasing context can improve prediction while simultaneously making the model harder to estimate from limited data.